# Software Design Document

**Project Title:** P2P Fault Detection Protocol with Decentralized Reporting

**Members:** Orhun İnan 2220356057 , Rıza Çakır 2220356059

# 1. Introduction

## 1.1 Purpose

The purpose of this Software Design Document (SDD) is to define the technical architecture and implementation details of the "P2P Fault Detection Protocol with Decentralized Reporting" project. This document aims to guide the development process by translating the high-level requirements from the project proposal into a concrete design plan. In this context, it details the C/C++ firmware for ESP32-based edge nodes, the configuration of redundant Raspberry Pi SBC (Single Board Computer) units, and the local network communication protocols that eliminate the need for a central server. Furthermore, the document provides technical specifications for fault tolerance, data structures, and hardware-software interfaces to ensure stable operation during physical leakage and fire detection scenarios.

## 1.2 Scope

**Software Products to be Produced:**

1. **Edge Node Firmware (C/C++):** Deployed on ESP32 microcontrollers. Handles real-time hardware interrupts from gas and flame sensors, manages local physical actuators (buzzers/LEDs), and executes the peer-to-peer (P2P) MQTT communication logic.
2. **Redundant Gateway Backend (Go / C#.NET):** Deployed across multiple Linux Single Board Computers (Raspberry Pi). This service acts as a cluster, passively logging MQTT traffic and providing a highly available REST/WebSocket API for the frontend.
3. **Monitoring Interface (Web/React):** A centralized dashboard providing a unified, real-time view of the industrial mesh network's status.

**System Capabilities (What it will do):**

- Continuously sample environmental data (gas concentrations and flame presence) at the edge.
- Autonomously trigger localized audio-visual alarms across a subset of peer nodes via MQTT without routing through a central application server.
- Maintain data logging and external dashboard connectivity even in the event of one or more Single Board Computer (SBC) hardware/network failures.
- Provide real-time telemetry and historical fault logs to remote administrators.

**System Limitations (What it will not do):**

- This software will not execute automated physical hazard suppression (e.g., triggering sprinkler systems or closing gas valves). It is strictly a monitoring, alerting, and reporting protocol.
- The protocol assumes an operational local 802.11 (Wi-Fi) layer; it does not currently implement a secondary physical transmission fallback if the local Wi-Fi router fails completely, though nodes will still attempt localized fallback routines.

### 1.3 Definitions, Acronyms, and Abbreviations

- **P2P:** Peer-to-Peer. In this context, nodes communicating directly via a shared messaging bus without a master controller processing the logic.
- **IoT:** Internet of Things.
- **SBC:** Single Board Computer (specifically Raspberry Pi in this deployment).
- **MQTT:** Message Queuing Telemetry Transport. A lightweight publish-subscribe network protocol.
- **Broker:** A server that receives all messages from clients and then routes the messages to the appropriate destination clients (e.g., Mosquitto).
- **ESP32:** A low-cost, low-power system on a chip with integrated Wi-Fi, functioning as the edge sensor node.
- **QoS (Quality of Service):** An agreement between the sender and receiver of a message regarding the guarantees of delivering a message in MQTT.

### 1.4 References


1. *Espressif Systems, "ESP32 Technical Reference Manual," [Online]. Available: https://www.espressif.com*
2. *OASIS, "MQTT Version 3.1.1 Standard," 2014.*
3. *F. C. Gartner, "Fundamentals of Fault-Tolerant Distributed Computing," ACM Computing Surveys, 1999.*
4. *Raspberry Pi Foundation. (2023). Raspberry Pi Documentation - Computers & Configuration. Retrieved from https://www.raspberrypi.com/documentation/*


## 2. System Overview

Industrial environments face risks from localized faults like LPG leaks. Traditional safety monitoring networks utilize centralized architecture: all sensor nodes send data to a central server, which evaluates the data and trigger alarms. This introduces a critical point of failure. If the server crashes or the link to it is severed, the facility loses both monitoring and alarming capabilities.

This project introduces a **Decentralized Architecture**. By distributing the logic directly to the ESP32 sensor nodes and leveraging the publish-subscribe model of MQTT, nodes establish a localized "mesh-like" structure. If "Node A" detects a problem, it immediately publishes an emergency payload. "Node B" and "Node C" in the physical vicinity, subscribed to Node A's topic, receive this payload and trigger their local alarms. This bypasses any central server.

A cluster of Raspberry Pi SBCs monitors this local network. Running redundant backend services, these SBCs log the events to a database and serve the web-based dashboard. If one SBC fails due to power loss, the others continue the reporting, guaranteeing that off-site personnel maintain visibility.[1]

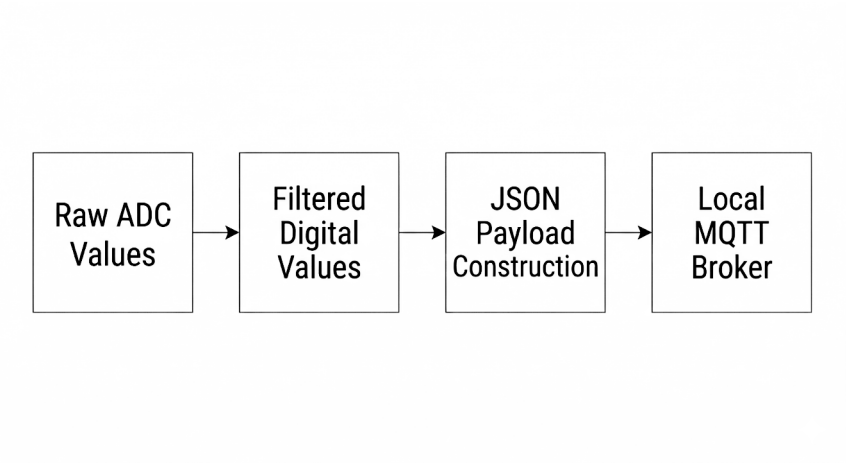
## 3. System Design

## 3.1 Design Method

1. **State-Machine (Edge Layer):** The ESP32 firmware is designed using C++ object. Hardware interactions are abstracted into classes. The core logic operates as an Event-Driven Finite State Machine (FSM) transitioning between `NORMAL`, `WARNING`, and `CRITICAL_FAULT` states based on analog sensor thresholds and incoming peer MQTT events.[1]
2. **Microservices & Active-Active Redundancy (Gateway Layer):** The backend relies on a stateless architecture. Multiple SBC instances run identical backend binaries. They do not rely on a single master database; instead, each maintains its own synchronized local database to ensure the API remains accessible regardless of individual device health.

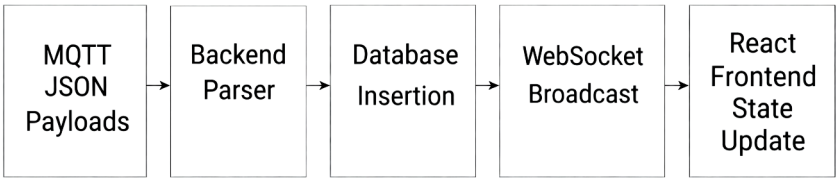
## 3.2 Decomposition Description

1. **Perception & Actuation Layer (ESP32 Firmware):** * *Control Flow:* Interrupt Service Routines (ISRs) for problem detection and continuous polling loops for analog sensors.
    - *Data Flow:*




    -
2. **Communication & Middleware Layer (MQTT Brokers):**
    - *Control Flow:* Topic-based routing.
    - *Data Flow:* Receiving payloads from nodes and distributing them to peer nodes and listening to SBC backend services.
3. **Reporting & Application Layer (SBC Backend & Dashboard):**
    - *Control Flow:* HTTP Request routing, WebSocket connection management, Database write/read coordination.

- *Data Flow:*




# 4. Component Description

## 4.1 Component Identifier: ESP32

1. **Type:** * *Logical:* C++ Class Architecture (e.g., `SensorPoller`, `AlertManager`, `MqttTransceiver`).
    - *Physical:* Compiled binary flashed to ESP32 microcontrollers.
2. **Purpose:** Fulfills localized, autonomous fault detection and P2P peer alerting.
3. **Function:** * Initializes GPIO pins for flame sensors (digital interrupt), gas sensors (analog read), buzzers (PWM), and LEDs (digital write).
    - Maintains a persistent Wi-Fi and MQTT connection with keep-alive pings.
    - Samples environmental data every 500ms. If a threshold is breached, it publishes a serialized JSON payload to `facility/zoneX/alerts`.
    - Listens to `facility/zoneX/alerts`; if an alert from a peer is received, it evaluates proximity logic and triggers local hardware alarms (Buzzers/LEDs) to warn workers in the immediate area.[3]
4. **Subordinates:** * `PubSubClient` (MQTT networking).
    - `WiFi.h` (Network stack).
    - `ArduinoJson` (Payload serialization/deserialization).
5. **Dependencies:** Requires an active local Wi-Fi LAN and at least one reachable IP address hosting the Mosquitto MQTT broker.
6. **Interfaces:**
    - *Input (Hardware):* GPIO Analog (0-4095) for gas, GPIO Digital (High/Low) for flame.
    - *Input (Network):* Subscribes to MQTT topic: `facility/+/alerts`.
    - *Output (Hardware):* GPIO PWM signals to piezoelectric buzzers.

    - *Output (Network):* Publishes to MQTT topics: `facility/zone[ID]/node[ID]/telemetry` and `facility/zone[ID]/alerts`.
7. **Data:** * `internal_state`: Enum `{ IDLE, FAULT_DETECTED, PEER_ALARM_ACTIVE }`.
    - `threshold_gas`: Integer defining the safe PPM limit.
    - `json_payload`: e.g., `{"node_id": "esp_01", "type": "gas_leak", "val": 3500, "timestamp": 1711829392}`.

## 4.2 Component Identifier: SBC_CLUSTER

1. **Type:** * *Logical:* Messaging Middleware / Publish-Subscribe Router.
    - *Physical:* Eclipse Mosquitto daemon running on Raspberry Pi OS.
2. **Purpose:** Facilitates the decentralized P2P routing without requiring custom backend routing logic.
3. **Function:** Authenticates ESP32 nodes and backend services. Routes incoming payloads from publishing nodes strictly to nodes and backends that have explicitly subscribed to those topics. Enforces Quality of Service (QoS 1) to ensure critical alerts are not dropped due to temporary network jitter.[2]
4. **Subordinates:** Linux TCP/IP Stack.
5. **Dependencies:** Relies on multiple SBCs acting in a bridged or parallel configuration to prevent broker failure from bringing down the localized messaging mesh.
6. **Interfaces:** * TCP Port 1883 (Standard MQTT).
7. **Data:**
    - In-memory topic trees and client subscription tables.
    - Retained message storage for late-joining nodes to immediately know the current alert state.

## 4.3 Component Identifier: BACKEND

1. **Type:** * *Logical:* High-Availability Application and Database Service.
    - *Physical:* Compiled Go or C#.NET executable service running as a systemd daemon on multiple SBCs.
2. **Purpose:** Bridges the local edge network to the external world, ensuring historical data persistence and uninterrupted dashboard serving.
3. **Function:** * Acts as an MQTT client, subscribing to `#` (all facility topics) to passively log all telemetry and fault events.
    - Parses incoming JSON and writes time-series data to a local database.
    - Exposes a REST API for the frontend dashboard to fetch historical data.
    - Maintains WebSocket connections to the web dashboard to push real-time alerts.
    - *Redundancy Logic:* Nodes broadcast "heartbeats" to each other via a backend-specific MQTT topic. External network traffic (Dashboard requests) is load-balanced or DNS round-robined across the active SBCs. If SBC-1 dies, SBC-2 is already logging identical data and can seamlessly serve the client.[4]

4. **Subordinates:** * Database Engine (e.g., SQLite or PostgreSQL).
    - HTTP/WebSocket Web Server Library (e.g., Go `net/http` or ASP.NET Core).
5. **Dependencies:** Relies on the SBC_BROKER_CLUSTER for incoming data.
6. **Interfaces:**
    - *Input:* MQTT Subscriptions.
    - *Output:* HTTP GET/POST endpoints (e.g., `/api/v1/nodes/status`, `/api/v1/alerts/history`). WebSocket stream at `/ws/realtime`.
7. **Data:**
    - `NodeHealthTable`: Tracks the last-seen timestamp of every ESP32 to detect offline nodes.
    - `EventLogTable`: Structured relational data mapping `EventID`, `Timestamp`, `NodeID`, `FaultType`, and `Severity`.

## 4.4 Component Identifier: DASHBOARD

1. **Type:** * *Logical:* Single Page Application (SPA).
    - *Physical:* Minified HTML/CSS/JavaScript bundle served to a web browser.
2. **Purpose:** Provides the human-machine interface (HMI) for remote facility administrators.
3. **Function:** * Connects to the REDUNDANT_BACKEND via HTTP to load the initial facility layout and historical fault logs.
    - Establishes a persistent WebSocket connection to receive instantaneous UI updates.
    - Renders a topological map or list of ESP32 nodes, visually distinguishing their states (Green = Normal, Red/Blinking = Fault).
    - Displays an active system health module showing which SBC gateways are currently online, verifying redundancy status.
4. **Subordinates:** * React.js (Component Framework).
    - Charting Library (e.g., Chart.js or Recharts) for gas concentration trends.
5. **Dependencies:** Requires a modern web browser and network access to the SBCs' IP addresses/domain.
6. **Interfaces:**
    - Consumes the REST API and WebSocket streams defined in the REDUNDANT_BACKEND.
7. **Data:**
    - `ApplicationState`: Redux or Context store holding the live representations of `nodes[]`, `active_alerts[]`, and `sbc_cluster_status[]`.